

Lecture 19: Structured Streaming II (Concepts)

DSL351 / DSP352: Big Data Analytics

Amit Kumar Dhar

IIT Bhilai

April 8, 2026

Aggregation not on Event Time

Can be of two types

- **Global Aggregation:** Aggregations across all the data in the stream
For example: `val runningCount = sensorReadings.groupBy().count()`
- **Grouped aggregations:** Aggregations within each group or key present in the data stream.
For example: `val baselineValues = sensorReadings.groupBy("sensorId").mean("value")`

What is allowed

- **All built-in aggregation functions:** `sum()`, `mean()`, `stddev()`, `countDistinct()` etc.
- **Multiple aggregations computed together:**
For example:

```
1      val multipleAggs = sensorReadings
2      .groupBy("sensorId")
3      .agg(count("*"), mean("value").alias("baselineValue"),
4      collect_set("errorCode").alias("allErrorCodes"))
5
```

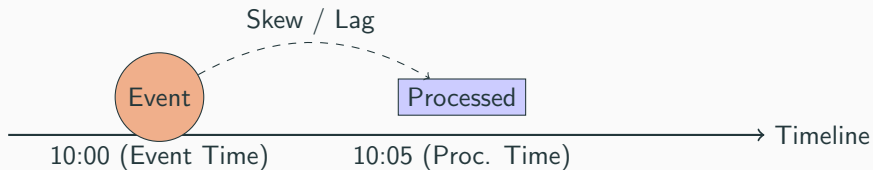
- **User-defined aggregation functions**

Windows on Event Time

Event Time vs. Processing Time

- **Processing Time:** When Spark gets the data.
- **Event Time:** When the data was created (timestamp in data).
- Critical for accurate analytics (e.g., user activity at midnight).

Processing vs. Event Time Diagram



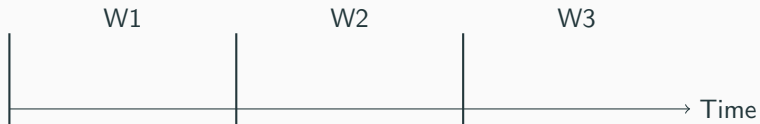
Windowing Concepts

- Aggregating data over time intervals.
- **Tumbling Windows:** Fixed-size, non-overlapping.
- **Sliding Windows:** Overlapping, fixed-size.

1. Tumbling Windows

- No gaps, no overlaps.
- Example: "Count requests every 5 minutes".
- Windows: [00:00, 00:05), [00:05, 00:10).

Tumbling Windows Visual



Tumbling Windows Code: Python

```
1 from pyspark.sql.functions import window
2
3 windowedCounts = df \
4     .groupBy(
5         window(df.timestamp, "10 minutes"),
6         df.category
7     ) \
8     .count()
```

2. Sliding Windows

- Windows overlap.
- Requires **Window Duration** and **Slide Duration**.
- Example: "10-minute window, starts every 5 minutes".

Sliding Windows Visual



Sliding Windows Code: Scala

```
1 val windowedCounts = df
2   .groupBy(
3     window(col("timestamp"), "10 minutes", "5 minutes")
4   )
5   .count()
```

Handling Late Data and Watermarks

The Problem of Late Data

- Network glitches cause events to arrive out of order.
- How long should Spark keep the state of a window open?
- If we wait forever, memory overflows.

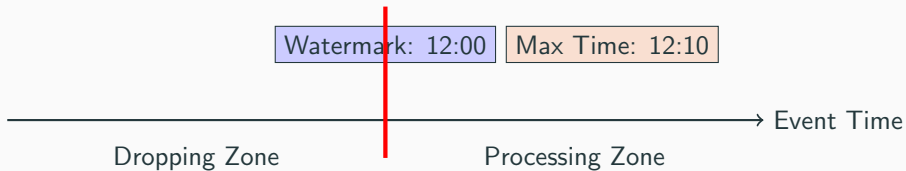
Watermarking: The Solution

- **Watermark:** A moving threshold defining the limit of "late data".
- Calculation: $\text{Watermark} = \max(\text{Event Time seen}) - \text{Threshold}$.
- Data older than the watermark is dropped.

Watermarking Step-by-Step

1. System sees event @ 12:05. Threshold = 10m.
2. Watermark = 11:55.
3. Event arrives @ 11:58. → **Keep** (within 10m).
4. Event arrives @ 11:50. → **Drop** (older than 11:55).

Watermarking Visual



Watermarking Code: Python

```
1 df.withWatermark("timestamp", "10 minutes") \  
2   .groupBy(window("timestamp", "10 minutes")) \  
3   .count()
```

State Cleanup with Watermarks

- When `Watermark > Window End Time`, Spark deletes the window state.
- This prevents memory OOM.
- Only works for **Append** and **Update** modes.

Cumulative Aggregations

- What if you want a global count?
- Use `Update` mode without windows.
- Spark maintains the state indefinitely.

Structured Streaming vs. DStreams (RDDs)

DStreams	Structured Streaming
RDD based	DataFrame based
Manual state handling	Automatic state management
Processing time only	Native Event Time/Watermarks
No SQL optimization	Catalyst/Tungsten optimized

Checkpoints and Fault Tolerance

- Essential for production.
- `.option("checkpointLocation", "/path").`
- Stores query progress as JSON and HDFS files.
- Allows the job to resume exactly where it left off.

- **Offsets:** Which Kafka messages were read.
- **Commits:** Which batches were completed.
- **State:** Binary snapshots of window aggregates.

Production Checklist

1. Always use watermarking for stateful queries.
2. Always define a checkpoint location.
3. Monitor the Spark UI (Streaming tab).
4. Ensure sink is idempotent.
5. Match trigger interval to your latency needs.

- **Infinitely growing state:** Forgetting watermarks.
- **Mixing sources:** Joins between two streams (Class 20).
- **Source data retention:** Kafka topic data deleted before Spark reads it.

Summary of Key Concepts

- Unbounded Table = Stream as a Table.
- Micro-batch = Core engine.
- Event Time = Truth in Big Data.
- Watermark = Memory management for late data.

Questions?